Ejercicio Demostrativo

Grafos — Unidad 6

Auxiliar: Sergio Sebastian Sandoval Ruiz

Tiempo estimado: 60 min

5 minutos para Valentía y 55 minutos de desarrollo académico

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas
Segundo Semestre 2026
Laboratorio de Estructuras de Datos C

1. Marco Formativo

1.1 Valores

Valentía	Actuar correctamente aunque sea difícil o arriesgado. En la práctica, preguntar cuando un recorrido no se comprende, reconocer un error al marcar visitados y presentar una solución para revisarla con honestidad, aunque exista temor a equivocarse.

1.2 Competencias

Tipo de competencia	Redacción
Competencia general	Implementa estructuras de datos lineales y no lineales mediante programación orientada a objetos para desarrollar soluciones eficientes.
Competencia específica	Representa un grafo simple no dirigido con una matriz de adyacencia, implementa operaciones sobre aristas y verifica recorridos BFS y DFS. Interpreta grados, alcanzabilidad, distancias y la reconstrucción de caminos mediante padres.

2. Palabras Clave

Grafo: Conjunto de vértices y aristas que representa entidades relacionadas.

Vértice y arista: Entidad del modelo y conexión entre dos entidades.

Adyacencia: Relación directa entre extremos de una arista.

Grado: Número de aristas incidentes; en este grafo simple coincide con la cantidad de vecinos.

Matriz de adyacencia: Arreglo de conexiones para cada pareja de vértices.

BFS: Recorrido en anchura que utiliza una cola y explora por niveles.

DFS: Recorrido en profundidad que utiliza una pila o llamadas recursivas.

Padre y distancia: Predecesor de descubrimiento y número de aristas desde el origen en BFS.

3. Resumen

La demostración de la unidad 6 desarrolla definición y características, operaciones principales, tipos de Grafos y recorridos BFS y DFS. Se construye un grafo pequeño con seis vértices y se utiliza el mismo conjunto de conexiones para comparar ambos recorridos.

El auxiliar presenta una traza antes de ejecutar el programa y contrasta la salida con resultados calculados a mano. El estudiante debe explicar los cambios de estado y justificar las condiciones de cada operación.

4. Contenido

4.1 DESCRIPCIÓN DEL EJERCICIO

Representar V = {A,B,C,D,E,F} y las aristas AB, AC, BD, CE, DE y EF mediante una matriz booleana. Consultar conexiones y grados, ejecutar BFS y DFS desde A y retirar la arista CE al terminar las trazas. Los índices 0 a 5 corresponden a las etiquetas A a F.

4.2 OBJETIVO DE LA DEMOSTRACIÓN

> Demostrar la diferencia entre una conexión directa y un camino, mantener la simetría de una arista no dirigida y explicar los estados de BFS y DFS. Reconstruir una ruta mínima en cantidad de aristas a partir de los padres calculados por BFS.

4.3 RECURSOS NECESARIOS

- Java JDK 11 o superior, editor o IDE y terminal con javac y java.
- Dos archivos en el mismo directorio: `Grafo.java` y `Main.java`.
- Diapositivas de Grafos de la unidad 6.
- Pizarrón o papel para trazar índices, estados y resultados.

Distribución del tiempo

Actividad	Minutos
Valentía y participación	5
Conceptos y traza inicial	15
Implementación en Java	20
Pruebas en consola	10
Contraste y cálculo manual	5
Resultados y cierre	5

4.4 DESARROLLO PASO A PASO

El grafo del programa es simple, no dirigido y no ponderado. No admite lazos y una conexión repetida no crea otra arista. El número de vértices se fija al construirlo; esta versión no incluye inserción o eliminación de vértices.

Paso 1 — Valor y participación (5 min)

Actuar correctamente aunque sea difícil o arriesgado. En la práctica, preguntar cuando un recorrido no se comprende, reconocer un error al marcar visitados y presentar una solución para revisarla con honestidad, aunque exista temor a equivocarse.

Solicitar una propuesta antes de mostrar la respuesta. Comparar los procedimientos con la misma entrada y cuidar que la revisión permita comprender el error y corregirlo.

Paso 2 — Conceptos y traza (15 min)

Dibujar el grafo base y escribir sus vecinos en orden alfabético. La matriz se recorre por índices crecientes, por lo que ese orden coincide con A, B, C, D, E y F. Los vértices pueden tener varios caminos entre sí y el grafo contiene un ciclo.

Vértice	Vecinos	Grado
A	B, C	2
B	A, D	2
C	A, E	2
D	B, E	2
E	C, D, F	3
F	E	1

Qué observar:

- Hay 6 vértices y 6 aristas. La suma de grados es 12, igual a dos veces el número de aristas.
- A y F no son adyacentes, pero F es alcanzable desde A.
- Conectar u-v actualiza dos celdas; el contador de aristas no cuenta dos veces la relación.
- La matriz favorece la consulta directa, pero ocupa $O(V^2)$ posiciones.

Paso 3 — Implementación (20 min)

Crear `Grafo.java`. La matriz fija las conexiones de cada pareja. Los métodos validar, conectar, desconectar, existe, grado y aristas permiten observar las operaciones principales antes de iniciar los recorridos.

java

```
public class Grafo {
    private final boolean[][] ady;
    private int[] padre, distancias;
    public Grafo(int n) {
        if (n < 1) throw new IllegalArgumentException("Tamano
invalido");
        ady = new boolean[n][n];
    }
    private void validar(int u) {
        if (u < 0 || u >= ady.length)
            throw new IndexOutOfBoundsException("Vertice invalido");
    }
    public void conectar(int u, int v) {
        validar(u); validar(v);
        if (u == v) throw new IllegalArgumentException("Lazo no
admitido");
        ady[u][v] = ady[v][u] = true; padre = null;
    }
    public void desconectar(int u, int v) {
        validar(u); validar(v); ady[u][v] = ady[v][u] = false; padre =
null;
```

```
    }
    public boolean existe(int u, int v) {
        validar(u); validar(v); return ady[u][v];
    }
    public int grado(int u) {
        validar(u); int g = 0;
        for (boolean conexion : ady[u]) if (conexion) g++;
        return g;
    }
    public int aristas() {
        int e = 0;
        for (int u = 0; u < ady.length; u++)
            for (int v = u+1; v < ady.length; v++)
                if (ady[u][v]) e++;
        return e;
    }
    public int[] bfs(int s) {
        validar(s); int n = ady.length;
        int[] distancia = new int[n], cola = new int[n];
        padre = new int[n]; distancias = distancia;
        for (int i = 0; i < n; i++) {
```

Continuación de la implementación:

La segunda parte completa BFS y DFS. BFS utiliza un arreglo como cola y marca un vértice al descubrirlo mediante una distancia distinta de -1. El método camino usa los padres del último BFS y devuelve Sin ruta si el destino fue inalcanzable. Cambiar una arista invalida esos padres.

java

```
            distancia[i] = -1; padre[i] = -1;
        }
        int frente = 0, fin = 0;
        cola[fin++] = s; distancia[s] = 0;
        while (frente < fin) {
```

```
            int u = cola[frente++];
            System.out.print((char)('A'+u)+" ");
            for (int v = 0; v < n; v++) {
                if (ady[u][v] && distancia[v] == -1) {
                    distancia[v] = distancia[u]+1; padre[v] = u;
                    cola[fin++] = v;
                }
            }
        }
        System.out.println(); return distancia;
    }
    public String camino(int destino) {
        validar(destino);
        if (padre == null)
            throw new IllegalStateException("Ejecutar BFS primero");
        if (distancias[destino] < 0) return "Sin ruta";
        String ruta = "";
        for (int u = destino; u != -1; u = padre[u])
            ruta = (char)('A'+u)+(ruta.isEmpty() ? "" : "-"+ruta);
        return ruta;
    }
    public void dfs(int s) {
        validar(s); dfs(s,new boolean[ady.length]);
        System.out.println();
    }
    private void dfs(int u, boolean[] visto) {
        visto[u] = true; System.out.print((char)('A'+u)+" ");
        for (int v = 0; v < ady.length; v++)
            if (ady[u][v] && !visto[v]) dfs(v,visto);
    }
}
```

Copiar las dos partes consecutivamente en el archivo indicado. No agregar una segunda declaración de clase ni omitir las llaves de cierre. La prueba del siguiente paso utiliza esta misma implementación.

Paso 4 — Pruebas en consola (10 min)

Crear `Main.java`. Dibujar el estado inicial y ejecutar primero los dos recorridos sin cambiar las conexiones. La eliminación de CE ocurre después para que sus resultados no se mezclen con la traza del grafo base.

java

```java
public class Main {
    public static void main(String[] args) {
        Grafo g = new Grafo(6);
        g.conectar(0,1); g.conectar(0,2); g.conectar(1,3);
        g.conectar(2,4); g.conectar(3,4); g.conectar(4,5);
        System.out.println("Aristas: "+g.aristas());
        System.out.println("Grado E: "+g.grado(4));
        System.out.println("Existe A-F: "+g.existe(0,5));
        System.out.print("BFS: "); int[] dist = g.bfs(0);
        System.out.print("Distancias:");
        for (int v : dist) System.out.print(" "+v);
        System.out.println();
        System.out.println("Camino A-F: "+g.camino(5));
        System.out.print("DFS: "); g.dfs(0);
        g.desconectar(2,4);
        System.out.println("Aristas sin C-E: "+g.aristas());
        System.out.println("Existe C-E: "+g.existe(2,4));
    }
}
```

Compilar y ejecutar:

```
javac Grafo.java Main.java
java Main
```

Salida esperada:

```
Aristas: 6
Grado E: 3
Existe A-F: false
BFS: A B C D E F
Distancias: 0 1 1 2 2 3
Camino A-F: A-C-E-F
DFS: A B D E C F
Aristas sin C-E: 5
Existe C-E: false
```

BFS reconstruye A-C-E-F con tres aristas. DFS muestra orden de descubrimiento, no un camino que deba copiarse completo para llegar a F. En una variante que guarde sus padres, el camino de DFS al destino puede tener más aristas.

Paso 5 — Traza de BFS y DFS (5 min)

BFS inicia con cola [A]. Tras procesar A, la cola es `[B,C]`; después de B, `[C,D]`; después de C, `[D,E]`; después de D, [E]; después de E, [F]; y después de F, []. E no vuelve a encolarse desde D porque ya se descubrió desde C.

DFS avanza A-B-D-E-C. En C no hay vecinos nuevos, de modo que retorna a E y después visita F. Su descubrimiento es A,B,D,E,C,F y su finalización C,F,E,D,B,A. El orden depende de los vecinos, no solo de las conexiones.

Para un grafo dirigido, las celdas no tienen que ser simétricas y cada recorrido respeta la dirección. Para un grafo ponderado, BFS no garantiza el costo mínimo si los pesos difieren. Estas variantes se comparan conceptualmente; el código mantiene el modelo no dirigido sin pesos.

Vértice	Padre en BFS	Distancia desde A
A	Sin padre	0
B	A	1
C	A	1
D	B	2

Vértice	Padre en BFS	Distancia desde A
E	C	2
F	E	3

Pedir una explicación del resultado, no únicamente su lectura. El cálculo manual debe utilizar exactamente los datos y el orden indicados. Si cambia una entrada, se deben actualizar también las expectativas de la prueba.

Observaciones Importantes

- Marcar al encolar evita insertar un vértice varias veces y permite que una cola de V posiciones sea suficiente.
- Reiniciar las marcas en cada recorrido independiente. No reutilizar las marcas de BFS en DFS.
- Después de cambiar conexiones, ejecutar BFS otra vez antes de pedir un camino.
- Un origen solo cubre los vértices alcanzables. Un grafo desconectado necesita nuevos inicios para una exploración completa.

4.5 Tabla Comparativa

TABLA COMPARATIVA

Operación	Costo en este código	Motivo
Consultar o cambiar arista	O(1)	Acceso a dos celdas
Calcular grado	O(V)	Recorrer una fila
Contar aristas	$O(V^2)$	Revisar la matriz
BFS o DFS completo	$O(V^2)$	Escanear filas
Memoria auxiliar de recorrido	O(V)	Cola, padres o pila
Matriz de adyacencia	$O(V^2)$	Almacenamiento del grafo

4.6 Resultado Esperado

El estudiante debe obtener las trazas previstas y diferenciar el orden de visita de una ruta por padres. Con listas de adyacencia, BFS y DFS pueden costar O(V+E); esta implementación utiliza matriz y debe analizarse con $O(V^2)$.

Verificación	Resultado
Simetría	Existe(u,v) coincide con Existe(v,u)
BFS desde A	Distancia a F igual a 3
DFS desde A	A,B,D,E,C,F
Eliminar CE	Quedan cinco aristas

4.7 Aplicación Práctica

Los grafos representan redes de equipos, conexiones de transporte y relaciones entre entidades. BFS permite hallar rutas con el menor número de pasos en grafos sin pesos; DFS permite explorar en profundidad y estudiar alcanzabilidad. Elegir el tipo de grafo exige definir dirección, pesos y repetición de conexiones.

Paso 6 — Resultados y cierre (5 min)

Pedir que un estudiante proponga el siguiente vértice y que otro revise la regla aplicada. Si la propuesta es incorrecta, corregirla mostrando el estado de la cola o la recursión. Relacionar esa participación con la valentía para preguntar y reconocer errores.

5. Recomendaciones al siguiente tutor

Dificultades previstas:

- Los errores previsibles son modificar una sola celda de una arista no dirigida, marcar tarde en BFS y confundir la lista completa de visita con un camino. También debe diferenciarse un vértice sin padre porque es el origen de otro que quedó inalcanzable.

Recomendaciones para futuras sesiones:

- Usar el mismo grafo y el mismo orden de vecinos durante toda la demostración.

- Mostrar la cola después de cada paso de BFS y los retornos de las llamadas en DFS.
- Agregar un séptimo vértice aislado en una prueba separada y comprobar distancia -1 y Sin ruta.
- Explicar la diferencia de costos entre matriz y listas sin atribuir al código una representación que no utiliza.

6. Referencias

1. Sedgewick, R., y Wayne, K. Algorithms, 4th edition. Undirected Graphs. https://algs4.cs.princeton.edu/41graph/
2. Sedgewick, R., y Wayne, K. Directed Graphs. https://algs4.cs.princeton.edu/42digraph/
3. Material de clase. Grafos, unidad 6. Laboratorio de Estructuras de Datos C, Escuela de Ingeniería en Ciencias y Sistemas, USAC, 2026.